HRMS — Phase 2

Teams, employees, invites and audit — development and test report

Field	Value
Phase	Phase 2 — Teams and employees (Phases 0–4 are the MVP)
Status	Backend verified by execution. Frontend typechecked and built; browser flow not verified.
Backend tests	153 passed, 0 failed (pytest 9.1.1, Django 5.2.18, SQLite)
Frontend	tsc --noEmit exit 0 · vite build 455 kB JS / 139 kB gzip
Database	SQLite for development. PostgreSQL not exercised.
Date	2026-10-19
Handbook basis	docs/Handbook.pdf, Phase 2 (pages 7–8)

What this document is

A record of everything done for Phase 2: the models and services that were written, the permissions and scoping rules, the React pages, the commands that were run with their real output, the defects and toolchain failures found and fixed, and — explicitly — what has **not** been verified. Where something was not executed it is labelled as such rather than described as working.

Summary

- **Phase 2 is complete against the Handbook's checklist.** Teams and employees have role-scoped CRUD, employee creation sends a one-time invite instead of a password, people are deactivated rather than deleted, and every change writes one audit row.
- **The backend test suite grew from 67 to 153 passing tests** and now covers the permission matrix, invite flow, deactivation, filters and the audit rows.
- **The whole flow was driven over real HTTP against a running server:** HR creates a person, the invite is accepted by that person, an employee sees only their own record, a manager sees only their team, a deactivated person cannot sign in, and audit rows carry the right actor.
- **Ten significant problems were found and fixed.** The most important were a wrong status code for hard deletes, audit rows that could miss email/role changes, manager scoping that would have leaked, missing privilege guards, and a seeded admin that could not actually use the Django admin.
- **Two things were deliberately left out of Phase 2:** no audit API/page (not in the Phase 2 build list) and no new frontend dependencies (the dialog and select are hand-built on existing primitives).
- **Not verified:** a browser click-through, PostgreSQL, ruff (blocked by the sandbox), and the Django admin forms beyond opening their list pages.

Phase 2 acceptance criteria (from the Handbook)

Handbook requirement	Evidence
HR creates an employee who logs in through the invite link	Live HTTP run plus 8 invite tests; the console email contains the one-time link.
Each role sees only what it should, enforced by the API	153-test suite includes the full role matrix; out-of-scope records return 404, never 403.

Handbook requirement	Evidence
Audit rows exist for the changes	Tests assert exactly one row per create/update/deactivate with the actor; live run showed create by HR and updates by HR and by the employee.
Watch out: employee edits a record or field they should not	Own record is phone-only (403 on any other field); another record is 404. Tested.
Watch out: duplicate audit rows	One save is one row, and a save that changes nothing writes nothing. Tested.

1. Scope and deliverables

Phase 2 goal (from the Handbook)

One place for people data: teams, employees, invites and an audit trail. Deliverables: Team and Employee models with CRUD and role-scoped querysets; creating an employee also creates a login and an invite link to set a password; deactivate instead of delete; AuditLog rows written automatically on create, update and delete; and React pages for the employee list (search, filters, pagination), the employee form, teams, and accepting an invite.

Delivered

Item	Where	Status
Team model, unique name, optional employee lead	apps/organization	Done, migrated
Employee model (user, names, phone, job title, team, manager, start date, type, status)	apps/employees/models.py	Done, migrated
One-time EmployeeInvite (UUID token, 7-day expiry)	apps/employees/models.py	Done, migrated
Role-scoped employee and team viewsets	apps/core/scoping.py, views.py	Done, tested
Create employee = login + record + invite email	apps/employees/services.py	Done, tested
Accept-invite endpoint and page that set a password	employees/views.py, AcceptInvitePage	Done, tested
Deactivate and reactivate instead of delete	services.py, viewset actions	Done, tested
AuditLog written by signals with request-user middleware	apps/audit	Done, tested
Employee list with search, team/status/role filters, pagination	EmployeesPage.tsx	Done, typechecked
Employee detail and create/edit form	EmployeeDetailPage, EmployeeFormPage	Done, typechecked
Teams page with create/edit dialog	TeamsPage.tsx	Done, typechecked
Accept-invite React page	AcceptInvitePage.tsx	Done, typechecked
seed_users extended with teams and employee links	accounts/management/commands	Done, idempotent
docs/PERMISSIONS.md with the endpoint/role matrix	docs/PERMISSIONS.md	Done
README updated for Phase 2	README.md	Done

Explicitly not in Phase 2

- No audit API or audit page. Rows are written and are readable in the Django admin; the Phase 2 build list stops at writing them. This is recorded in docs/PERMISSIONS.md.
- No invite resend, no bulk import, no CSV export, no avatars, no employee self-service profile page.
- No leave models or endpoints — Phase 3.
- No new npm packages. shadcn/ui components were written by hand on top of the packages that were already installed (Radix dropdown/label/slot, Tailwind, cva). A native select and a small custom dialog cover the new UI needs.

Files created and changed

```
backend/apps/organization/
  models.py  serializers.py  views.py  urls.py  admin.py  apps.py
```

```
  migrations/{0001_initial}.py
backend/apps/employees/
  models.py  serializers.py  services.py  views.py  urls.py  admin.py  apps.py
  migrations/{0001_initial,0002_initial}.py
backend/apps/audit/
  models.py  signals.py  middleware.py  context.py  admin.py  apps.py
  migrations/{0001_initial}.py
backend/tests/
  test_employees.py  test_teams.py  test_invites.py  test_audit.py
  test_auth.py (seed/admin tests added)   conftest.py (factories added)
backend/
  config/settings/base.py  config/urls.py
  apps/core/permissions.py  apps/core/scoping.py
  apps/accounts/management/commands/seed_users.py
  requirements.txt (ruff declared)
  build_phase02_pdf.py (this report's generator)
frontend/src/
  api/{client,queries,employees,teams,types}.ts  lib/{query,format,useDebouncedValue}.ts
  components/ui/{table,badge,skeleton,select,dialog}.tsx
  components/common/{states,Pagination}.tsx  components/form/Field.tsx
  components/employees/StatusBadge.tsx
  pages/employees/{EmployeesPage,EmployeeDetailPage,EmployeeFormPage}.tsx
  pages/teams/TeamsPage.tsx  pages/auth/AcceptInvitePage.tsx
  App.tsx  components/layout/navItems.ts  components/layout/Sidebar.tsx
  pages/DashboardPage.tsx
docs/PERMISSIONS.md  README.md  .gitignore
```

2. Data model and migrations

Model	Fields	Notes
organization.Team	name (unique), lead (nullable FK to Employee, SET_NULL)	Lead is a person, not a login, because manager scoping and approvals hang off it.
employees.Employee	user (one-to-one), first_name, last_name, phone, job_title, team (FK, SET_NULL), manager (self FK, SET_NULL), start_date, employment_type, status	One record per person; the login lives on User. Never hard-deleted.
employees.EmployeeInvite	user (one-to-one), token (UUID, unique), created_at, expires_at, accepted_at	One active invite per user. Token is single-use and expires after 7 days.
audit.AuditLog	actor (nullable FK), action, model, object_id, changes (JSON), created_at	Append-only; actor is null for system changes such as the seed.

The circular foreign key, and how the migrations were split

Team.lead points at Employee, while Employee.team points back at Team. A single migration cannot create both tables. Django resolved this itself by splitting the work, and the resulting order had to be read and checked rather than assumed:

```
employees.0001   create Employee (without team/user), EmployeeInvite (without user)
organization.0001  create Team (lead FK to employees.employee)
employees.0002   add Employee.team, Employee.user, EmployeeInvite.user

manage.py migrate output on the dev database:
  audit.0001_initial        OK
  employees.0001_initial    OK
  organization.0001_initial OK
  employees.0002_initial    OK
```

The migrations are generated, reviewed and committed; nothing destructive is in them. makemigrations --check --dry-run reports no pending changes.

Database-level rules in force

- Team.name is unique in the database, and a case-insensitive duplicate is rejected by the serializer (the database's unique index is case-sensitive on both SQLite and PostgreSQL).
- User.email is unique (from Phase 1) and the manager lowercases the whole address; Employee.user is one-to-one, so one login cannot own two employee records.
- EmployeeInvite.token is unique and generated with uuid4; one invite per user is enforced by the one-to-one.
- Deleting a user cascades to the employee record and its invite. Deleting a team sets members' team to NULL; deleting a manager sets their reports' manager to NULL. People are not deleted by any API path.
- Status and employment type are constrained to their TextChoices at the application layer; status is read-only on the API so the record and the login cannot disagree.
- A person cannot be their own manager: rejected by the serializer. This is application-level rather than a database CHECK, because Django's CHECK API differs across the Django 5.0–5.2 range this project allows.

3. The invite flow

HR never types a password for someone else. Creating a person creates an inactive login with an unusable password, an employee record and a one-time invite, all inside one transaction, and emails the link.

```
1. POST /api/v1/employees/            HR or admin, with email, names, start date, team, role
2. services.create_employee()          one transaction:
     User(email, role, is_active=False, unusable password)
     Employee(user, ...)
     EmployeeInvite(token=uuid4, expires_at=now+7d)
     send_mail(...)                     console backend in development
3. Email contains:  <FRONTEND_URL>/accept-invite?token=<uuid>
4. The person opens the React page and chooses a password.
5. POST /api/v1/auth/accept-invite/   public, CSRF-protected: {token, password}
6. Serializer validates token (exists, unused, unexpired), employee still active,
   and runs Django's password validators.
7. Service sets the password, sets is_active=True, marks the invite accepted, and
   the view signs the person in. The React page goes straight to the dashboard.
```

Security properties

- The token is a UUID4 (122 random bits), stored unique, single-use, and expires after seven days.
- Unknown, used and expired tokens all return the same message: “This invite link is invalid or has expired.” The endpoint does not help someone guess tokens or enumerate accounts.
- A deactivated employee's outstanding invite is refused, so an old link cannot switch an inactive account back on.
- The acceptance endpoint is public but CSRF-protected, exactly like login and password reset, because it changes state.
- Passwords go through Django's validators; a weak password is a field error, not a 500.
- The audit trail deliberately records no employee change for acceptance: the Employee row did not change, only the login did. Deactivation, which does change the record, is audited.

Edge cases covered by tests

Case	Expected	Test
Token reused after acceptance	400, generic message	test_token_can_only_be_used_once
Token expired	400, generic message	test_expired_token_is_rejected
Token never existed	400, generic message	test_unknown_token_is_rejected
Employee deactivated before accepting	400	test_deactivated_employee_cannot_accept
Weak password	400 with a password field error	test_weak_password_is_a_field_error
No CSRF token	403	test_accept_requires_a_csrf_token
Accept, then log in normally	200 as the new employee	test_accepted_password_works_on_the_login_page

4. The audit trail

Audit rows are written by signals and only by signals, so one save is one row. Nothing in a view or service inserts audit rows by hand, which is what keeps the count predictable.

How a row is produced

- pre_save loads the row currently stored under the instance's primary key and takes a JSON-safe snapshot of every concrete field except the primary key. A new instance has no snapshot.
- post_save compares that snapshot with the new values. Create writes {"after": {...}}, update writes {"changes": {field: {"before": ..., "after": ...}}}, and a save with no differences writes nothing at all.
- The service can attach extra changes that live on the User row (email, role) to the instance; the signal folds them into the same single row instead of creating a second one.
- post_delete writes {"before": {...}} from the instance that was deleted.
- The actor comes from a ContextVar set by CurrentUserMiddleware, which runs after AuthenticationMiddleware. Management commands and other request-less writes leave the actor null rather than guessing.

```
create:  {"after": {"first_name": "Vera", "team": 2, "status": "active", ...}}
update:  {"changes": {"job_title": {"before": "QA Engineer", "after": "Staff QA"}}}
delete:  {"before": {"name": "Sales", "lead": null}}

actor: the signed-in user, or null for the seed and other system writes.
model: the lower-case app label, e.g. "employees.employee", "organization.team".
```

What is tracked today

Employee and Team. LeaveType and LeaveRequest join the same machinery in Phase 3 by being added to TRACKED_MODELS; no other change is needed. The audit log itself is not tracked, and User changes made directly in the Django admin are outside the employee audit trail.

Where to read it

Django admin at /admin/audit/auditlog/, read-only: no add, change or delete permission is granted. Only the seeded admin account is staff and superuser. There is no API or React page for the audit log yet; that was outside the Phase 2 build list and is recorded as a gap.

> **Why not audit every save?**
>
> A no-op PATCH (sending the same job title) should not look like a change. The signal skips it, and a test asserts that a PATCH with unchanged values creates no row. Bulk queryset .update() calls bypass signals by design; the application always saves model instances, which keeps the trail accurate.

5. Permissions and API

Scoping rules

Role	Employees	Teams
employee	own record only	no access (403)
manager	members of teams they lead, plus their own record	teams they lead (read-only)
hr	everyone, including inactive people	every team, read and write
admin	everyone, including inactive people	every team, read and write

Out-of-scope records are removed from the queryset, so a record outside your scope is a 404 rather than a 403. A 403 would confirm that the record exists. The scoping lives in apps/core/scoping.py (scope_employees, scope_teams) and fails closed for unknown roles.

Endpoints added

Method	Path	Allowed
GET	/employees/ (search, team, status, role, page)	all roles, scoped
GET	/employees/{id}/	all roles, scoped
POST	/employees/	hr, admin
PATCH / PUT	/employees/{id}/	hr, admin; employee on own record, phone only
DELETE	/employees/{id}/	nobody — 405 with a deactivate hint
POST	/employees/{id}/deactivate/	hr, admin (an admin target needs an admin)
POST	/employees/{id}/reactivate/	hr, admin (an admin target needs an admin)
GET	/teams/ (search)	manager, hr, admin; manager scoped to led teams
POST	/teams/	hr, admin
GET	/teams/{id}/	manager, hr, admin; scoped
PATCH / PUT	/teams/{id}/	hr, admin
DELETE	/teams/{id}/	hr, admin
POST	/auth/accept-invite/	public, CSRF-protected, one-time token

Field rules that tests enforce

- An employee PATCHing their own record may send only phone. Any other field, or phone mixed with another field, is a 403 with the message “Employees may only edit their own phone number.” It is not silently ignored.
- status is not writable through the API; deactivation and reactivation are explicit actions. This prevents an HR PATCH from leaving employee.status and user.is_active disagreeing.
- Only an admin can grant or remove the admin role. HR may assign employee, manager and hr, so it can still create a manager.

- Only an admin can deactivate or reactivate an admin account. Without that guard, HR could switch an admin off or back on.
- A manager cannot write employee or team data at all; the manager role can read its own team and nothing else.

Error shape

Every response still uses {"detail": str, "code": str, "fields": {}} from the Phase 1 exception handler. Validation failures populate fields with the field names; permission and 404 responses populate detail. This is what the React forms map onto inline errors.

6. Services, seed and demo data

Service	What it does	Transaction
create_employee(actor, data)	Checks the role grant, creates the inactive User, the Employee and the invite, then emails the link.	atomic
update_employee(employee, actor, data)	Applies User email/role changes and Employee fields; hands the User diff to the audit signal.	atomic
deactivate_employee(employee, actor)	Sets status inactive and the login inactive; refuses an admin target unless the actor is an admin.	atomic
reactivate_employee(employee, actor)	Restores both, with the same admin guard. Deactivation is not a dead end.	atomic
accept_invite(invite, password)	Sets the password, activates the login, marks the invite accepted.	atomic
send_invite_email(invite)	Builds the FRONTEND_URL link and sends it with Django's mail backend.	inside create

Demo company created by seed_users

Role	Email	Team	Manager
admin	admin@hrms.test	People Ops	—
hr	hr@hrms.test	People Ops (lead)	—
manager	manager@hrms.test	Engineering (lead)	—
employee	employee@hrms.test	Engineering	Meera Manager
employee	employee2@hrms.test	Engineering	Meera Manager

All five use the password DemoPass123!. The command is idempotent: it creates the teams before the people, creates the people with their team, then links managers and team leads in a second pass. Re-running it resets demo passwords, keeps the counts at 5/5/2, and does not duplicate rows.

The admin account, and the error that exposed a real bug

Only admin@hrms.test is staff. The Django admin at /admin/ is not the HRMS application; it is Django's own admin, and it is deliberately restricted to that account. A manager who visits it is signed in but not authorized, which is the message the browser shows. That part is correct behaviour.

The error was useful because it led to a real defect: the seed made admin@ staff but not superuser, and a staff user without model permissions opens an empty admin. The seed now grants both flags, two tests cover it (admin reaches the audit and employee list pages; a manager is redirected to the admin login), and the local database was re-seeded. Verified against the dev database: /admin/, /admin/audit/auditlog/, /admin/employees/employee/ and /admin/organization/team/ all return 200 for admin@hrms.test.

> **If you land on the admin as the wrong user**
>
> The admin login page offers “log in to a different account”, or open /admin/logout/ and sign in again. The email field needs the full address (admin@hrms.test), not the word admin. The React app at http://localhost:5173 is where the manager, HR and employee roles work.

7. Frontend implementation

Page	Route	Behaviour
Employees	/employees	Search (debounced) across name, email and job title; team, status and role filters; 20 per page with a count and page controls; role and status badges; table from 768px up, cards below that.
Employee detail	/employees/:id	Profile, team, manager, start date, employment type and access role; Edit, Deactivate (confirmation dialog) or Reactivate, depending on status and role.
New employee	/employees/new	react-hook-form + zod, inline errors, team and manager selects, role rules, start date; on success goes to the new profile.
Edit employee	/employees/:id/edit	Same form prefilled; role is locked when HR edits an admin, matching the API rule.
Teams	/teams	Search, member counts, lead names, create/edit dialog with an optional lead.
Accept invite	/accept-invite?token=...	Public page: password + confirmation, signs in on success, clear message for an invalid or expired link.

Data layer

- src/api/employees.ts and src/api/teams.ts hold every query and mutation: useEmployees, useEmployee, useCreateEmployee, useUpdateEmployee, useSetEmployeeStatus, useTeams, useCreateTeam, useUpdateTeam. Components never call fetch.
- Query keys are namespaced (["employees", ...], ["teams", ...]) so one invalidation refreshes lists and details after a mutation. Pagination uses keepPreviousData so changing pages does not blank the table.
- src/api/client.ts gained apiPatch; the existing error parsing turns {detail, code, fields} into ApiError, which forms map onto inline errors.

UI primitives added

- table, badge and skeleton follow the shadcn/ui idiom already in the project.
- select is a native <select> styled with Tailwind. It keeps the mobile picker, keyboard behaviour and 375px layout correct without another dependency.
- dialog is a small hand-written modal: portal, overlay click to close, Escape, body scroll lock, focus moved in and restored on close, and a Tab trap. The close button sits last in the DOM so the form's first field receives focus.
- Field, states (ErrorState, EmptyState) and Pagination keep the loading, empty, error and paging patterns identical across pages.

Routing and navigation

- Employees and Teams are now real, enabled sidebar items, filtered by role. Time off, Approvals and Audit log remain disabled “soon” entries until their phases.
- Routes: /employees and /employees/:id require admin, hr or manager; /employees/new, /employees/:id/edit and /teams require admin or hr; /accept-invite is public. The guards are a UX layer; the API remains the gate.
- The dashboard’s “coming next” card and the sidebar footer now describe Phase 3 rather than Phase 2.

> **Not verified in a browser**
>
> The React code typechecks and the production bundle builds, and every API call behind these pages was exercised over HTTP. But nobody has rendered the pages, opened a dialog, typed in a filter or looked at the layout at 375px. This section describes implemented behaviour, not observed behaviour.

8. Verification performed

Every command below was run; the output shown is the real output.

8.1 Static checks, migrations and seed

```
python manage.py check                        -> System check identified no issues (0 silenced)
python manage.py makemigrations --check --dry-run -> No changes detected
python manage.py migrate --noinput            -> audit.0001, employees.0001,
                                                 organization.0001, employees.0002 applied
python manage.py seed_users                   -> 5 users, 5 employees, 2 teams

database after the runs:
  users 5   employees 5   teams 2   audit rows 22   leftover verify users 0
```

8.2 Test suite

```
pytest -q -p no:cacheprovider   ->  153 passed in 2.76s   (exit 0)

Phase 1 had 67 tests. Phase 2 adds 86, across four new files.
```

8.3 What the tests cover

Area	Examples
Employee scoping	hr and admin see all; manager sees own team plus self; employee sees one record; another record is 404 for employee and for a manager outside the team; anonymous is rejected.
Employee writes	HR create with invite; HR cannot grant admin; admin can; manager and employee cannot create; duplicate email is a field error; required fields reported.
Self-edit	own phone 200; own job title 403 with the exact message; phone mixed with another field 403; another record 404; manager cannot PATCH a team member.
HR updates	job title, team and manager changed; email change updates the login; duplicate email rejected; self-manager rejected; role change to manager allowed, to admin refused.
Deactivation	login stops; idempotent and audited once; reactivate restores login; manager/employee refused; HR cannot deactivate or reactivate an admin.
Filters and pagination	team, status and role filters; search by name and email; unknown status is a 400; the paginated envelope is present.
Teams	scoping for every role; create with and without a lead; case-insensitive duplicate rejected; manager read-only; HR rename and delete; members keep their record; member counts; search.
Invites	one email with the token; cannot log in before accepting; acceptance activates and signs in; reused, expired, unknown and deactivated cases; weak password; CSRF.
Audit	create with actor and new values; update diff only; role change in the same row; no-op writes nothing; deactivation row; team create/update/delete rows; null actor without a request; login and reads write nothing.
Seed	roles, teams, employee links, idempotency, admin can use the Django admin, manager is redirected.

8.4 Live end-to-end run over HTTP

A throwaway client (removed afterwards) drove a real runserver on 127.0.0.1:8000 with a cookie jar and CSRF headers. Confirmed:

```
health 200; csrf 200; HR login 200
employees 200 (5 people); teams 200 (2 teams, Engineering lead and member count correct)
create employee 201; invite email printed with /accept-invite?token=...
search for the new address 200 (1 result)
```

```
accept invite 200; me 200; own employee list 200 (1 record); own detail 200
PATCH own phone 200; PATCH own job_title 403 ("Employees may only edit their own phone number.")
GET /teams/ as employee 403; DELETE own employee 405
logout 204; manager login 200; manager teams 200 (1); manager employees 200 (3, all Engineering)
manager GET the new hire (People Ops) 404
employee login 200; employees 200 (1 record)
HR login; deactivate 200; deactivated login 400; reactivate 200; login 200 again
audit: create by hr@, update by the new hire (phone), update by hr (deactivate),
       update by hr (reactivate)
```

8.5 Django admin access check

```
admin login: True
/admin/                          200
/admin/audit/auditlog/           200
/admin/employees/employee/       200
/admin/organization/team/        200

manager login, GET /admin/     302 -> /admin/login/ (expected: not staff)
```

8.6 Frontend

```
tsc --noEmit      -> exit 0
npm run build     -> 1733 modules transformed
                     dist/assets/index-*.css  20.43 kB (gzip 5.01 kB)
                     dist/assets/index-*.js  455.16 kB (gzip 139.05 kB)
                     built in 17.31s
```

9. Defects and errors solved

Ten application-level problems and a class of toolchain failures were found and fixed. Severity and how each was caught are stated, because some were design corrections rather than test failures.

9.1 Hard delete answered 403 instead of 405

Severity. Medium — wrong API contract; a client would read it as a permission problem rather than an unsupported operation.

Symptom. DELETE /api/v1/employees/{id}/ returned 403 after the permission class was changed to fail closed. The earlier test expected 405.

Root cause. DRF runs permission checks before it dispatches to the method handler. The fail-closed permission class refused the destroy action before the viewset could report that the method is not allowed.

Fix. The permission class now lets destroy through to an explicit destroy() that always raises MethodNotAllowed, and the view has no other delete path. The 405 detail tells the caller to deactivate instead.

Verified. test_delete_is_not_allowed passes, and the live run returned 405 with the deactivate message. No code path can hard-delete a person.

9.2 Email and role changes were invisible to the audit log

Severity. High — a role grant is one of the most important changes in the system; it must be recorded.

Symptom. Changing an employee's email or role produced no employee audit row, because both fields live on User, and the audit signal only snapshots Employee.

Root cause. The snapshot compared model fields only. Saving the User row never touches Employee, so the diff was empty and the signal correctly skipped it.

Fix. update_employee computes the User diff itself, attaches it to the employee instance as _audit_extra_changes, and the post_save signal folds it into the same row as any model-field changes. One action still means one row.

Verified. test_role_change_lands_in_the_same_row asserts one row containing both job_title and role changes, with before/after values.

9.3 Audit rows could duplicate or appear on no-op saves

Severity. Medium — a noisy or missing trail is worse than none; the Handbook explicitly warns about duplicate rows.

Symptom. Nothing prevented a save with unchanged values from writing an update row, and a second audit hook could have doubled rows for one change.

Root cause. The first design wrote a row in post_save without comparing against stored values.

Fix. pre_save stashes a snapshot; post_save diffs it and returns early when nothing changed. No service writes audit rows by hand, so exactly one path can produce a row.

Verified. test_a_save_that_changes_nothing_writes_nothing and the team update test (exactly one row) pass. Raw SQL .update() bypasses signals by design; the app saves instances.

9.4 Employees could send edits to fields they may not change

Severity. High — a validation-only approach would either silently ignore the attempt or require every future field to remember the rule.

Symptom. An employee PATCHing their own record with {"job_title": ...} had to be rejected, not accepted and ignored.

Root cause. Permission checks that consider only the action cannot express “only this field, only on your own row”.

Fix. The object-level permission compares the request body's keys with {phone} for the employee role and returns 403 with a specific message; the queryset still limits the reachable rows.

Verified. Own phone is 200; own job title is 403; phone mixed with another field is 403; another record is 404. All four are tested.

9.5 Manager scoping needed its own implementation

Severity. High — wrong scoping either leaks another team's data or hides a manager's own record.

Symptom. The Phase 1 helper scope_for_own_team filters on employee__team__lead__user, which works for records that hang off an employee (leave requests), but not for Employee itself, and a manager is also an employee.

Root cause. One generic strategy was being asked to serve two different shapes of data.

Fix. scope_employees filters on Q(team__lead__user=user) | Q(user=user) for managers and user=user for employees; scope_teams filters lead__user for managers. HR and admin get everything; everything else fails closed.

Verified. The matrix tests cover all four roles on both resources, and the live run confirmed a manager sees three Engineering people and gets 404 for a People Ops hire.

9.6 A seeded admin could not actually use the Django admin

Severity. High for operations — it made the documented way to inspect the audit table unusable, and it is the error the user hit in the browser.

Symptom. Signing in as manager@hrms.test at /admin/ showed “You are authenticated as manager@hrms.test, but are not authorized to access this page.” Investigating that surfaced the deeper issue: admin@hrms.test was staff but not superuser.

Root cause. seed_users set is_staff for the admin role but never is_superuser. A staff user with no model permissions can log in and then sees an empty admin index; the model pages are forbidden.

Fix. The seed now sets is_staff and is_superuser for the admin role and clears both for the others, then the local database was re-seeded. Two tests cover admin access and the manager redirect.

Verified. admin@ opens /admin/, the audit log, employees and teams (all 200); a manager is redirected to the admin login. The manager message itself is expected behaviour and is now explained in the README.

9.7 Privilege escalation through employee administration

Severity. High — HR manages people; it must not be able to create an admin or switch an admin off and on.

Symptom. HR could otherwise create an employee with role=admin, or deactivate/reactivate an admin account.

Root cause. Role assignment and deactivation were ordinary serializer/view operations with no rule about who may touch the admin role.

Fix. services.assert_can_assign_role refuses to grant or remove admin unless the actor is an admin, and deactivate/reactivate refuse an admin target unless the actor is an admin. HR may still assign employee, manager and hr.

Verified. Five tests: HR cannot create an admin, admin can; HR can create a manager; HR cannot change a role to admin; HR cannot deactivate or reactivate an admin.

9.8 Invite links had to be safe by construction

Severity. High — an invite is a password-setting credential.

Symptom. A token could have been reused, accepted after expiry, or used to revive a deactivated account; a weak password could have produced a 500.

Root cause. Acceptance initially looked only at whether the token existed.

Fix. The serializer rejects missing, accepted, expired and deactivated-employee cases with one generic message, runs Django's password validators and converts their errors to a field error; the service applies the change in a transaction and the view signs the person in.

Verified. Eight invite tests plus the live accept-and-log-in run; reuse, expiry, unknown token, deactivation, weak password and missing CSRF are all covered.

9.9 Circular foreign key across two apps

Severity. Medium — it blocks migrations if handled naively.

Symptom. Team.lead points at Employee and Employee.team points at Team, so neither app's initial migration could own both tables.

Root cause. A genuine model-level cycle, not an ordering mistake.

Fix. Let Django split it: employees.0001 creates the models without the cross-app fields, organization.0001 adds Team with lead, employees.0002 adds team, user and the invite's user. The order was reviewed and the migrations committed.

Verified. migrate applied all four cleanly, and makemigrations --check reports no pending changes.

9.10 Toolchain failures in the sandbox

Severity. Process — no application impact, but it limited what could be verified.

Symptom. (a) npm run build failed with spawn EPERM while esbuild started its helper process. (b) pip install ruff failed with permission denied writing a wheel metadata file in the temp directory, twice, including inside the workspace. (c) The resulting pip temp directories could not be deleted. (d) npm run typecheck printed a PowerShell $LASTEXITCODE error from npm.ps1 even though tsc succeeded.

Root cause. (a) The Windows sandbox blocks Node child processes that capture output through pipes; esbuild uses one. (b) The same confinement prevented the installer's temp writes. (c) The sandbox's deny entry on DeleteSubdirectoriesAndFiles blocks removing subdirectories, which is the intended confinement, not a fault. (d) A quirk of the npm PowerShell shim under the harness.

Fix. (a) The build was rerun once with wider access and succeeded. (b) No lint run was possible; ruff is declared in requirements.txt for the next session. (c) The empty .tmp/pip-* directories are left in place and .tmp/ is gitignored. (d) tsc was invoked directly (node_modules/.bin/tsc.cmd) and reported exit 0.

Verified. The production bundle was built and its sizes recorded; the test suite and typecheck are unaffected. The missing lint is listed as a gap, not glossed over.

9.11 Bugs in the throwaway verification script

Recorded because it shows the live run was real and iterative, not a transcript: the first pass expected 200 from logout (Django returns 204); the second asserted exactly five employees while a previous partial run had left a throwaway person behind, so the script now cleans leftovers before it starts; and the third assumed a “Sales” team that exists only in the test fixtures, not in the seeded company (the seed has People Ops). Each was a defect in the script, fixed and rerun, not in the application.

10. What is not verified

Item	Status
React pages in a browser	NOT DONE. Bundle builds, APIs exercised over HTTP, but no page has been rendered or clicked.
375px layout and dialogs	NOT DONE. Code-only; responsive classes and the custom dialog are unobserved.
PostgreSQL	NOT DONE. Everything ran on SQLite. The models and migrations use no Postgres-specific feature yet, but they have not been applied to a server.
ruff lint	NOT RUN. Not installed; installing was blocked by the sandbox. Declared in requirements.txt.
Frontend lint and tests	NOT AVAILABLE. npm run lint and npm run test do not exist; ESLint and Vitest were never set up in Phase 1.
Django admin forms	PARTIAL. List pages open for the admin; editing through the admin was not exercised.
Audit API or page	NOT BUILT. Outside the Phase 2 build list; rows are visible read-only in Django admin.
Invite resend	NOT BUILT. A lost invite currently means HR has no UI to reissue one.
Employee self-service profile page	NOT BUILT. The API allows an employee to read their own record and edit their phone; there is no page for it yet.
Phase 3	NOT STARTED.

Known limitations to carry forward

- The teams page requests up to 100 teams and has no pagination UI; the manager dropdown in the employee form is likewise limited to 100 colleagues.
- Manager scoping is defined as “teams you lead”. A manager who is a member of a team they do not lead does not see that team; this matches the approvals model and is documented in PERMISSIONS.md.
- User rows edited directly in the Django admin are not part of the employee audit trail.
- The audit actor is request-scoped; management-command writes are correctly attributed to nobody.
- Employee.status is read-only on the API, so a data fix must go through the actions (or the Django admin, which is deliberately read-only for the status).
- docs/HANDBOOK.md is still absent; the Agent instructions in docs/AGENTS.md point at it, but only the PDF exists.

11. Reproduce the phase

```
cd D:\HRMS\backend
.venv\Scripts\python.exe manage.py check
.venv\Scripts\python.exe manage.py migrate
.venv\Scripts\python.exe manage.py seed_users
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe manage.py runserver

# second terminal
cd D:\HRMS\frontend
npm run typecheck
npm run dev
```

Demo script (5 minutes)

- Sign in at http://localhost:5173 as hr@hrms.test / DemoPass123!.

- Employees: filter by Engineering, search for a name, then create a person (team People Ops, no manager).
- Copy the invite link printed in the Django console and open it; choose a password.
- You are now signed in as that person. Their Employees page shows only their own record.
- Sign in as manager@hrms.test: Employees shows only Engineering, and Teams is not in the sidebar.
- Sign in as hr@, open the new hire, deactivate them; a login attempt now fails. Reactivate.
- Sign in at http://127.0.0.1:8000/admin/ as admin@hrms.test / DemoPass123! and open Audit log.

> **Bottom line**
>
> Phase 2 is complete against the Handbook's checklist and verified by 153 tests plus a real HTTP session: teams and people with role-scoped access, invites that replace passwords, deactivation instead of deletion, and an audit trail with exactly one row per action. The defects that mattered most — a wrong delete status code, an audit log blind to role changes, manager scoping, missing privilege guards, and an admin account that could not open the admin — are fixed and covered. The honest gaps remain the browser, PostgreSQL and lint.